KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Password Vault and Crendential Storage System

Submitted by

Roshini.S.B

25BCS249

Under the Guidance of

Dharaneesh.S

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Password Vault and Credential Storage System "** submitted in partial fulfillment of the requirements for the course [**Java Programming]** is a **record of my original work** carried out under the guidance of **[Dharaneesh.s]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

Roshini.S.B

(Your Name)

CERTIFICATE

This is to certify that the project entitled **"Password Vault and Credential Storage System"** is a **bona fide work** carried out by **[Roshini.S.B] [B.Sc(CS)-1C]**, [**COMPUTER SCIENCE]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Java Programming]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

In the modern digital era, individuals and organizations interact with numerous online platforms such as banking systems, social media, email services, and enterprise applications. Each of these platforms requires authentication, typically in the form of usernames and passwords. As the number of accounts increases, managing multiple credentials becomes increasingly complex and challenging. Users often resort to insecure practices such as reusing passwords, writing them down, or storing them in unsecured digital formats, which significantly increases the risk of unauthorized access and cyber threats.

The Password Vault and Credential Storage System is developed to address these challenges by providing a centralized and secure solution for managing user credentials. The system allows users to store multiple usernames and passwords securely under a single master password. This reduces the burden of remembering multiple credentials while improving overall security.

The application is implemented using Java programming language and follows object-oriented programming (OOP) principles. It utilizes core Java concepts such as classes, objects, ArrayList, loops, and conditional statements to ensure efficient data handling and system functionality. The system includes features like user authentication, credential storage, retrieval, and a menu-driven interface for easy navigation.

Although the current system is console-based, it provides a strong foundation for future enhancements such as encryption, database integration, and graphical user interfaces. Overall, this project demonstrates the practical application of programming concepts in solving real-world problems related to digital security and data management.

1. INTRODUCTION

The rapid advancement of technology has led to an exponential increase in the number of online services used by individuals. From social networking platforms to financial applications, users are required to create and maintain multiple accounts. Each account requires a unique and strong password to ensure security. However, remembering multiple complex passwords is a difficult task for most users.

As a result, users often adopt unsafe practices such as using simple passwords, reusing the same password across multiple platforms, or storing passwords in insecure locations. These practices make systems vulnerable to cyber-attacks such as hacking, phishing, and data breaches. Therefore, there is a strong need for a secure and efficient system that can manage passwords effectively.

The Password Vault System is designed to solve this problem by providing a centralized repository for storing user credentials. Instead of remembering multiple passwords, the user only needs to remember a single master password. This system not only simplifies password management but also enhances security by reducing exposure to common threats.

The project also serves as an educational tool for understanding how programming can be used to develop secure applications. It demonstrates the use of Java and object-oriented programming concepts in building a real-world solution.

1.1 Objective

The primary objective of this project is to design and implement a secure and efficient password management system. The system aims to simplify the process of storing and retrieving credentials while ensuring basic security.

Specific objectives include:

To develop a Java-based password vault system

To implement a secure authentication mechanism using a master password

To store multiple credentials in an organized and structured format

To enable easy and quick retrieval of stored credentials

To apply object-oriented programming concepts such as encapsulation and modularity

To minimize the risks associated with password mismanagement

To create a user-friendly interface for smooth interaction

1.2 Overview

The Password Vault System is a menu-driven console application that provides users with an interactive interface to manage their credentials. The system is designed to be simple, efficient, and easy to use.

The overall workflow of the system includes:

User registration using a master password

Authentication of the user during login

Display of menu options after successful login

Options to add new credentials or view existing ones

Continuous operation until the user chooses to exit

The system uses the ArrayList data structure to dynamically store credentials. Each credential consists of:

Platform or service name

Username

Password

The use of ArrayList allows the system to handle multiple entries efficiently without requiring fixed memory allocation. This ensures flexibility and scalability.

1.3 Features

The system offers several important features:

Secure Authentication: Access is restricted using a master password

Credential Storage: Multiple credentials can be stored in a structured format

Data Retrieval: Stored credentials can be easily accessed when needed

User-Friendly Interface: Simple menu-driven navigation

Efficient Processing: Fast execution using optimized data structures

Scalability: Can handle multiple entries dynamically

2. PROGRAM ANALYSIS

2.1 Problem Definition

Managing passwords manually presents several challenges:

Users struggle to remember multiple complex passwords

Weak or repeated passwords reduce security

Storing passwords in plain text increases vulnerability

Manual tracking is inefficient and error-prone

High chances of data breaches and unauthorized access

These issues highlight the necessity for a secure and automated password management system.

2.2 Project Overview

The Password Vault System provides a comprehensive solution to the identified problems. It introduces a structured approach to credential management by offering:

A secure login mechanism

Organized storage of credentials

Easy retrieval of stored information

Reduced dependency on memory

The system ensures that users can safely manage their credentials without compromising security.

2.3 Scopes and Limitations

Scope:

- Personal password management
- Educational projects and demonstrations
- Small-scale applications
- Limitations
- Console-based interface limits user experience
- No encryption for stored passwords
- No persistent storage (data lost after program exit)
- Limited security compared to real-world systems

3. SYSTEM DESIGN

The system is designed using a modular approach to improve organization and maintainability. Each module is responsible for a specific function.

3.1 Modules

1. Authentication Module

The Authentication Module is responsible for ensuring that only authorized users can access the system. It acts as the first layer of security.

- **User Registration:**
  Allows a new user to create an account by setting a master password. This password is required every time the user logs in.
- **Secure Password Storage (Basic Level):**
  The master password is stored in a secure manner (for example, using simple encryption or hashing techniques in basic implementations). This prevents direct exposure of the password.
- **Login Validation:**
  When the user attempts to log in, the system compares the entered password with the stored password. If it matches, access is granted; otherwise, access is denied.
- **Unauthorized Access Prevention:**
  The system restricts access if incorrect credentials are entered multiple times. This helps protect sensitive stored data from misuse.
- **Session Control (Optional Enhancement):**
  Once logged in, the system maintains a session until the user logs out or exits, ensuring continuous secure access.

2. Vault Module

The Vault Module is the core component of the system, responsible for storing and managing user credentials.

- **Credential Storage:**
  Stores details such as platform name (e.g., Gmail, Facebook), username, and password in a structured format.

- **Dynamic Data Handling:**
  Uses an ArrayList to store multiple credential objects dynamically, allowing flexibility in adding or removing data.
- **Add New Credentials:**
  Users can add new account details easily. Each entry is stored as a separate object in the list.
- **Retrieve Credentials:**
  Users can view stored credentials when needed. The system fetches and displays data from the ArrayList.
- **Search Functionality (Optional):**
  Users can search for a specific platform or username to quickly locate stored credentials.
- **Update and Delete (Enhancement):**
  The module can be extended to allow users to modify or delete existing credentials.
- **Data Protection (Basic Level):**
  Passwords can be masked or partially hidden when displayed to improve security.

3. User Interface Module

This module provides interaction between the user and the system. It ensures that the application is easy to use and user-friendly.

- **Menu Display:**
  Shows different options such as Add Credential, View Credentials, Search, and Exit.
- **User Input Handling:**
  Accepts input from the user using tools like Scanner in Java.
- **Navigation Control:**
  Guides users through different operations step-by-step, ensuring they understand what to do next.

- **Input Validation:**
  Checks whether the entered data is valid (e.g., non-empty fields, correct menu choices).
- **Feedback Messages:**
  Displays success or error messages (e.g., “Credential added successfully” or “Invalid choice”).
- **Looping Mechanism:**
  Keeps the application running until the user chooses to exit, allowing multiple operations in a single session.

4. Exit Module

The Exit Module ensures that the application terminates safely and properly.

- **Safe Termination:**
  Stops the execution of the program when the user selects the exit option.
- **Resource Management:**
  Closes open resources such as input streams (e.g., Scanner) to prevent memory leaks.
- **Session End:**
  Ends the user session securely, ensuring no sensitive data remains exposed.
- **Final Message Display:**
  Optionally displays a closing message such as “Thank you for using the Password Manager System.”

4. METHODOLOGY

The development of the Password Management System follows a structured and systematic methodology to ensure clarity, efficiency, and reliability. Each phase plays a crucial role in building a robust application.

4.1. Problem Analysis

This phase focuses on identifying the issues in existing password management practices and understanding user needs.

- **Existing Problems:**
  - Users often store passwords in unsafe places such as notebooks or plain text files.
  - Difficulty in remembering multiple passwords.
  - High risk of data breaches and unauthorized access.
- **Requirement Identification:**
  - Secure storage of credentials.
  - Easy retrieval of saved passwords.
  - Simple and user-friendly interface.
  - Basic authentication mechanism for protection.
- **Outcome:**
  A clear understanding of system requirements and the need for a secure and efficient password manager.

4.2. System Design

In this phase, the system architecture is planned and divided into manageable modules.

- **Modular Design Approach:**
  - Authentication Module
  - Vault Module
  - User Interface Module
  - Exit Module
- **Design Considerations:**
  - Simplicity and ease of use
  - Data organization using object-oriented principles
  - Separation of concerns for better maintainability
- **Data Structure Design:**
  - Use of ArrayList to store credential objects dynamically.
  - Each credential contains platform name, username, and password.

- **Outcome:**
  A blueprint of how the system will function and how components will interact.

4.3. Program Development

This phase involves the actual coding and implementation of the system using Java.

- **Programming Language:**
  Java is used due to its object-oriented features and platform independence.
- **Key Components:**
  - **Classes:**
    Used to define entities such as User and Credential.
  - **Methods:**
    Implement functionalities like login validation, adding credentials, and displaying data.
  - **ArrayList:**
    Provides dynamic storage, allowing flexible addition and retrieval of credentials.
  - **Scanner Class:**
    Used for taking user input from the console.
- **Coding Approach:**

    - Modular coding for better readability
    - Reusable methods to reduce redundancy
    - Proper naming conventions for clarity
- **Outcome:**
  A functional password management system ready for testing.

4.4. Testing and Validation

This phase ensures that the system works correctly under different scenarios.

- **Testing Scenarios:**
    - **Login Testing:**
      Valid and invalid password attempts are checked.
    - **Data Entry Testing:**
      Adding single and multiple credentials.
    - **Data Retrieval Testing:**
      Verifying whether stored credentials are displayed correctly.
    - **Input Validation:**
      Handling incorrect inputs such as invalid menu choices.
- **Error Handling:**
    - Prevents system crashes due to invalid inputs.

    - Displays appropriate error messages.

- **Outcome:**
  A reliable and error-free system that performs as expected.

4.5. Result and Documentation

This phase evaluates the final output and prepares proper documentation.

- **System Performance:**
    - Successfully stores and retrieves credentials.
    - Provides secure access through authentication.
    - Maintains smooth user interaction.
- **Documentation Includes:**
    - System design and architecture
    - Code explanation
    - Testing procedures
    - User guide
- **Outcome:**
  A complete and well-documented project ready for submission and future improvements.

4.6. Algorithm

Step-by-step procedure of the system:

1. Start the program
2. Prompt user to register a master password
3. Store the password securely
4. Ask user to login
5. Validate entered password
6. If valid, display menu options
7. Perform selected operation (Add/View/Search credentials)
8. Repeat operations until user chooses to exit
9. Terminate the program

4.7. Flowchart

Start

↓

Register Password

↓

Login

↓

Password Validation

↓

Menu Display

↓

User Operation

↓

Repeat? ---- Yes ----> Back to Menu

↓ No

End

5. SOURCE CODE EXPLANATION

The program begins by importing required classes such as ArrayList and Scanner. The Credential class is defined to store user credentials including website, username, and password.

In the main method, the program first allows the user to create a master account by entering a username and password. These values are stored and later used for login verification.

After registration, the system prompts the user to log in. The entered username and password are compared with the stored values using conditional statements. If the credentials do not match, access is denied and the program terminates. If they match, the user is granted access to the vault.

An ArrayList is then created to store credential objects dynamically. The program enters a loop that continuously displays menu options to the user.

When the user selects the option to add credentials, the system asks how many credentials to add. It then uses a loop to collect details such as website, username, and password. Each set of details is stored as a new Credential object and added to the ArrayList.

When the user selects the option to view credentials, the system checks if the vault is empty. If it is empty, a message is displayed. Otherwise, all stored credentials are displayed using a loop.

If the user selects the exit option, the program displays a closing message and terminates the loop. Finally, the Scanner object is closed to release system resources.

Credential Class:

The Credential class is used to store individual account details. It contains three variables: website, username, and password. A constructor is used to initialize these values when a new credential object is created. Each object represents one set of login information.

SecureVault Class (Main Class):
This is the main class that controls the entire application. It contains the main method where execution begins. It handles user registration, login validation, menu display, and all vault operations such as adding and viewing credentials.

Object-Oriented Programming:
The system is designed using classes and objects. A separate class is used to represent each credential, which improves code organization and reusability. This approach helps in maintaining and extending the system easily.

ArrayList for Dynamic Storage:
An ArrayList is used to store credentials dynamically. It allows the system to store multiple entries without a fixed size limit. Users can add any number of credentials, and the list automatically grows as needed.

Conditional Statements for Decision Making:
Conditional statements such as if-else are used to control the flow of the program. They are used for login validation, menu selection, and checking whether the vault contains any data.

Loops for Repeated Operations:
Loops such as while and for are used to perform repeated tasks. The while loop keeps the application running until the user chooses to exit, while the for loop is used to add multiple credentials.

Scanner for User Input:
The Scanner class is used to take input from the user through the console. It allows the system to read different types of data such as strings and integers.

6.INPUT AND OUTPUT SCREENS

Input

The system requires several inputs from the user to perform authentication and securely store credentials.

1. **Master Password**
    - This is the primary password set by the user during registration.
    - It is used for authentication whenever the user tries to log in.
    - The system verifies this password before granting access to stored data.
    - It ensures that only authorized users can view or manage credentials.
2. **Platform Name**
    - Refers to the name of the website or application (e.g., Gmail, Facebook, Instagram).
    - Helps the user identify which account credentials are being stored.
    - Acts as a key for organizing and retrieving stored data efficiently.
3. **Username**
    - The login ID or email associated with the selected platform.
    - It is stored along with the platform name for easy identification.
    - Helps users remember which username is used for different services.
4. **Password**
    - The secret password associated with the platform account.
    - Stored securely within the system (can be encrypted in advanced versions).
    - Used only for display or retrieval when required by the user.

Output

The system generates various outputs based on user actions and system operations.

1. **Login Success or Failure Messages**
    - If the entered master password is correct:
        - Displays: *"Login Successful"*
        - Grants access to the system features.
    - If incorrect:
        - Displays: *"Invalid Password"*
        - Denies access and may prompt retry.
2. **Stored Credential Details**
    - Displays saved information such as:
        - Platform name
        - Username
        - Password
    - Helps users retrieve forgotten credentials.
    - May include options to search, update, or delete entries.
3. **Menu Options and Prompts**
    - After login, the system displays a menu such as:
        - Add new credentials
        - View saved credentials
        - Update existing credentials
        - Delete credentials
        - Exit system
    - Prompts guide the user step-by-step for each operation.
    - Ensures smooth and user-friendly interaction.

7.DISCUSSION & CONCLUSION

7. RESULTS AND DISCUSSION

The system performs efficiently and successfully meets all the functional requirements defined during the design phase. It ensures smooth operation and reliable execution of all features.

- **Accurate Credential Storage**
  The system stores all user credentials such as platform name, username, and password without any data loss. The use of dynamic data structures like ArrayList ensures flexibility and proper data management.
- **Reliable Authentication System**
  The master password authentication mechanism works correctly by validating user input before granting access. This ensures that unauthorized users cannot access sensitive information.
- **Simple and Effective User Interface**
  The system provides a user-friendly, menu-driven interface. Clear instructions and prompts guide the user, making the system easy to use even for beginners.

7.1 DISCUSSION

This project demonstrates how software solutions can significantly improve both security and efficiency in managing sensitive data such as passwords.

- It highlights the importance of **secure password management**, which is a critical issue in today's digital world where users handle multiple accounts.
- The system reduces the risk of forgetting passwords and minimizes unsafe practices like writing passwords on paper.
- It showcases the practical implementation of core programming concepts such as:
  - Object-Oriented Programming (OOP)

    - Data structures (ArrayList)
    - Conditional statements and loops
    - User input handling using Scanner
- The project also emphasizes the importance of designing systems that are both **secure and user-friendly**.

Overall, the project bridges the gap between theoretical knowledge and real-world application.

7.2 CONCLUSION

The Password Vault System successfully addresses the problem of managing multiple passwords securely and efficiently.

- It provides a **secure solution** through master password authentication.
- It offers a **simple interface** that allows users to store, retrieve, update, and delete credentials easily.
- The system ensures **efficient performance** with accurate data handling and quick access to stored information.

The project fulfills all its objectives and demonstrates the practical use of programming skills in solving real-life problems. It also serves as a strong foundation for developing more advanced security systems in the future.

FUTURE ENHANCEMENTS

The current system can be further improved by implementing the following features:

1. **AES Encryption for Password Security**
    - Encrypt stored passwords using Advanced Encryption Standard (AES).
    - Ensures that even if data is accessed, it remains unreadable without a decryption key.
2. **Database Integration for Persistent Storage**
    - Replace temporary storage (ArrayList) with databases like MySQL or SQLite.
    - Allows data to be stored permanently and retrieved across sessions.
3. **Graphical User Interface (GUI)**
    - Develop a GUI using Java Swing or JavaFX.
    - Enhances user experience with buttons, forms, and visual components.
4. **Automatic Password Generator**
    - Generate strong, random passwords with a mix of characters, numbers, and symbols.
    - Helps users create secure passwords easily.
5. **Cloud-Based Storage**
    - Store encrypted data on cloud platforms.
    - Enables access from multiple devices while maintaining security.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com